UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MORELOS

Instituto de Investigación en Ciencias Básicas y Aplicadas
Centro de Investigación en Ciencias
Lic. en Inteligencia Artificial

Detección de Noticias Falsas

Bayes ingenuo frente a regresión logística

Modelo generativo frente a modelo de máxima entropía

Minería de Texto

Dr. Jorge Hermosillo Valadez

Cuernavaca, Morelos, México — Noviembre 2026

César Mauricio Ocampo Ramírez
Rebecca Shaday Castillo Ortega
Kevin Vargas Flores

Índice

1. Descripción del proyecto **3**
- 1.1. Contexto y motivación . . . 3
- 1.2. Problema . . . 3
- 1.3. Corpus . . . 3
- 1.4. Organización del trabajo . . . 3

2. Fundamentos teóricos **4**
- 2.1. Clasificación probabilística y regla de Bayes . . . 4
- 2.2. Representación del texto . . . 4
- 2.3. Bayes ingenuo multinomial . . . 5
- 2.4. Regresión logística con regularización L_2 . . . 5
- 2.5. Máxima entropía . . . 6
- 2.6. Generativo frente a discriminativo . . . 6
- 2.7. Métricas de evaluación . . . 6
- 2.8. Umbral de decisión y costos . . . 7

3. Trabajo en clase (en papel) **7**
- 3.1. Ejercicio 1: Bayes ingenuo a mano . . . 7
- 3.2. Ejercicio 2: Bayes ingenuo es un clasificador lineal . . . 8
- 3.3. Ejercicio 3: umbral y costos . . . 9
- 3.4. Ejercicio 4: métricas en un problema desbalanceado . . . 9

4. Etapa 1: análisis, representación y *baseline* **10**
- 4.1. Exploración del corpus . . . 10
- 4.2. Partición de los datos . . . 13
- 4.3. Representación: bolsa de palabras . . . 14
- 4.4. *Baseline*: Bayes ingenuo multinomial . . . 14

5. Etapa 2: TF-IDF y regresión logística **16**
- 5.1. Representación TF-IDF . . . 16
- 5.2. Regresión logística con regularización L_2 . . . 16
- 5.3. Comparación de los modelos en Test . . . 17
- 5.4. Ablación: ¿efecto del modelo o de la representación? . . . 18
- 5.5. Efecto del umbral de decisión . . . 19

6. Etapa 3: atributos adicionales y ajuste del umbral **20**
- 6.1. Atributos de forma del texto . . . 20
- 6.2. Ajuste del umbral . . . 22

7. Experimentos de control e interpretación **24**
- 7.1. Control 1: palabras con mayor log-razón . . . 25
- 7.2. Control 2: etiquetas permutadas . . . 26
- 7.3. Control 3: dependencia de las marcas de fuente . . . 27

8. Debate: modelo generativo frente a modelo discriminativo **28**
8.1. Argumento a favor del modelo generativo (Bayes ingenuo) 28
8.2. Argumento a favor del modelo discriminativo (regresión logística) 28
8.3. Conclusión del debate 28

9. Conclusiones **29**
9.1. Limitaciones 29

A. Bitácora de contribuciones **31**

1. Descripción del proyecto

1.1. Contexto y motivación

Nuestro objetivo es que, a partir del texto, podamos distinguir una noticia falsa de una verdadera.

Comparamos dos modelos que representan filosofías distintas de la clasificación probabilística: el **modelo generativo** (Bayes ingenuo), que aprende cómo se *generan* los textos de cada clase, y el **modelo discriminativo** (regresión logística, equivalente a un modelo de máxima entropía [1]), que aprende directamente a *separar* las clases. Ambos producen una frontera de decisión lineal sobre las mismas representaciones del texto, por lo que la comparación aísla el efecto del método de estimación.

1.2. Problema

Dado el texto de una noticia, queremos predecir si es falsa o verdadera. La entrada del sistema es el texto de la noticia y la salida es una probabilidad $P(\text{fake} \mid \text{texto})$ junto con una decisión que obtenemos al compararla con un umbral. Nuestros objetivos son:

- comparar Bayes ingenuo multinomial con regresión logística regularizada con L_2;
- estudiar el efecto del umbral de decisión sobre los errores de cada tipo;
- evaluar si atributos adicionales de forma (longitud, dígitos, URLs y mayúsculas) mejoran el desempeño;
- comprobar mediante experimentos de control qué señales aprenden realmente los modelos.

1.3. Corpus

El corpus contiene **44,898 noticias** en inglés, con cinco columnas: `title`, `text`, `subject`, `date` y `label`. No hay valores nulos. Inicialmente hay 23,481 noticias falsas (52.3 %) y 21,417 verdaderas (47.7 %).

1.4. Organización del trabajo

Organizamos el trabajo en ejercicios en papel y tres etapas. Todo el código está en nuestro cuaderno de Colab [5].

- **Trabajo en clase** (Sección 3): cuatro ejercicios resueltos a mano.
- **Etapa 1** (Sección 4): análisis del corpus, representación mediante bolsa de palabras y *baseline* con Bayes ingenuo multinomial.
- **Etapa 2** (Sección 5): TF-IDF con regresión logística, comparación de modelos y estudio del umbral.

- **Etapa 3** (Sección 6): atributos adicionales y ajuste del umbral.
- **Experimentos de control** (Sección 7) y debate generativo frente a discriminativo (Sección 8).

La distribución del trabajo por integrante se encuentra en el Anexo A.

Nota sobre los ejercicios en papel. Los ejercicios de la Sección 3 usan el ejemplo de juguete del enunciado, formulado con mensajes de tipo *spam*/*ham*. Las ideas son exactamente las que aplicamos después a noticias; solo cambian los nombres de las clases (*spam*→*fake*, *ham*→*real*).

2. Fundamentos teóricos

2.1. Clasificación probabilística y regla de Bayes

Sea d un documento y $c \in \{\text{fake}, \text{real}\}$ su clase. Un clasificador probabilístico calcula $P(c \mid d)$ y elige la clase más probable o compara esa probabilidad con un umbral. La regla de Bayes establece

$$P(c \mid d) = \frac{P(d \mid c)\, P(c)}{P(d)},$$

donde $P(c)$ es la probabilidad *a priori* de la clase, $P(d \mid c)$ es la *verosimilitud* del documento si perteneciera a la clase c, y $P(d)$ es una normalización común a ambas clases que se cancela al compararlas.

2.2. Representación del texto

Bolsa de palabras (BoW). Fijamos un vocabulario V y representamos cada documento mediante el vector de conteos $x = (x_1, \ldots, x_{|V|})$, donde x_j es el número de veces que aparece la palabra j. Perdemos el orden de las palabras, pero conservamos qué palabras hay y cuántas veces. La matriz resultante es muy dispersa, por lo que la almacenamos en formato `scipy.sparse`.

TF-IDF. Los conteos crudos dan mucho peso a palabras muy comunes que no distinguen clases. TF-IDF reescala cada término t del documento d como

$$w_{t,d} = \mathrm{tf}_{t,d} \cdot \mathrm{idf}_t, \qquad \mathrm{tf}_{t,d} = 1 + \log \mathrm{count}(t,d) \;\; (\text{si count} > 0), \qquad \mathrm{idf}_t = \log \frac{1+N}{1+\mathrm{df}_t} + 1,$$

donde N es el número de documentos y df_t el número de documentos que contienen a t [3]. El logaritmo en el tf atenúa el efecto de repeticiones altas y el idf penaliza los términos que aparecen en muchos documentos. Después normalizamos cada vector con norma L_2.

2.3. Bayes ingenuo multinomial

Calcular $P(d \mid c)$ para un documento completo es inviable. Bayes ingenuo supone que, dada la clase, las palabras son *independientes*, de modo que

$$P(d \mid c) \;\propto\; \prod_{j=1}^{|V|} \theta_{j,c}^{x_j}, \qquad \theta_{j,c} = P(w_j \mid c).$$

La suposición es falsa en el lenguaje natural (hay fuertes correlaciones entre palabras), pero hace que el modelo dependa solo de conteos. Estimamos las probabilidades con suavizado aditivo

$$\theta_{j,c} = \frac{n_{j,c} + \alpha}{N_c + \alpha|V|},$$

donde $n_{j,c}$ es el conteo de la palabra j en la clase c, $N_c = \sum_j n_{j,c}$ y $\alpha > 0$; $\alpha = 1$ es el suavizado de Laplace. Sin suavizado, una palabra nunca vista en una clase daría probabilidad cero a cualquier documento que la contenga. En la práctica trabajamos con logaritmos para evitar *underflow*:

$$\log P(c \mid d) = \log P(c) + \sum_j x_j \log \theta_{j,c} + \text{constante}.$$

Bayes ingenuo es lineal. El log-cociente de posteriores es

$$\log \frac{P(\text{fake} \mid d)}{P(\text{real} \mid d)} = \underbrace{\log \frac{P(\text{fake})}{P(\text{real})}}_{b} + \sum_j x_j \underbrace{\log \frac{\theta_{j,\text{fake}}}{\theta_{j,\text{real}}}}_{w_j} = \mathbf{w}^T\mathbf{x} + b,$$

y $P(\text{fake} \mid d) = \sigma(\mathbf{w}^T\mathbf{x} + b)$ con $\sigma(z) = 1/(1+e^{-z})$. Cada peso w_j es una *log-razón*, lo que hace al modelo directamente interpretable.

2.4. Regresión logística con regularización L_2

La regresión logística tiene la misma forma funcional, $P(\text{fake} \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b)$, pero $\mathbf{w}$ y b se *aprenden* minimizando la entropía cruzada sobre los datos de entrenamiento:

$$J(\mathbf{w}, b) = -\sum_i \Big[y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big] + \frac{\lambda}{2}\|\mathbf{w}\|^2, \qquad p_i = \sigma(\mathbf{w}^T\mathbf{x}_i + b).$$

El término $\frac{\lambda}{2}\|\mathbf{w}\|^2$ es la **regularización** L_2: penaliza pesos grandes y limita el sobreajuste, algo especialmente necesario con decenas de miles de variables. En `scikit-learn` la fuerza de la regularización se controla con C, de modo que C pequeño equivale a regularización fuerte y C grande a regularización débil.

2.5. Máxima entropía

El principio de máxima entropía establece que, entre todos los modelos compatibles con lo observado en los datos, debemos elegir el que *menos supone*, es decir, el de mayor entropía [1]. Concretamente, exigimos que el valor esperado de cada atributo bajo el modelo coincida con el observado en los datos y, de todos los modelos que lo cumplen, tomamos el de mayor entropía condicional. La solución tiene la forma $P(y \mid x) \propto \exp(\mathbf{w} \cdot f(x, y))$, que para dos clases coincide con la regresión logística. A diferencia de Bayes ingenuo, no introduce el supuesto de independencia entre palabras.

2.6. Generativo frente a discriminativo

	Bayes ingenuo (generativo)	**Regresión logística (discriminativo)**
Qué modela	$P(d \mid c)$ y $P(c)$	$P(c \mid d)$ directamente
Obtención de $\mathbf{w}$	Conteo (forma cerrada)	Optimización de una pérdida
Supuesto clave	Independencia entre palabras dada la clase	Ninguno sobre la distribución de las palabras
Palabras correlacionadas	Cuenta la misma evidencia varias veces	Reparte el peso entre ellas

Tabla 1: Comparación conceptual entre los dos modelos.

Ng y Jordan [4] muestran que el modelo generativo tiende a alcanzar su error asintótico con menos datos, mientras que el discriminativo suele alcanzar un error asintótico menor. En la Sección 8 contrastamos estas ideas con nuestros resultados.

2.7. Métricas de evaluación

Tomamos la clase *Fake* como positiva. Con VP (verdaderos positivos), FP (noticias reales marcadas como falsas), FN (noticias falsas que pasaron) y VN:

$$\text{Exactitud} = \frac{VP + VN}{VP + VN + FP + FN}, \qquad \text{Precisión} = \frac{VP}{VP + FP},$$

$$\text{Exhaustividad} = \frac{VP}{VP + FN},$$

$$F_1 = \frac{2 \cdot \text{Precisión} \cdot \text{Exhaustividad}}{\text{Precisión} + \text{Exhaustividad}} = \frac{2\,VP}{2\,VP + FP + FN}.$$

El **AUC-ROC** es el área bajo la curva que grafica la tasa de verdaderos positivos contra la de falsos positivos al variar el umbral; equivale a la probabilidad de que una noticia falsa elegida al azar reciba un puntaje mayor que una verdadera elegida al azar (0.5 es el azar y 1 es perfecto). Como referencia, un clasificador que declara siempre *Real* obtiene en el conjunto de prueba una exactitud de 0.5484.

2.8. Umbral de decisión y costos

Sea $p = P(\text{fake} \mid d)$, C_{FP} el costo de marcar como falsa una noticia real y C_{FN} el de dejar pasar una falsa. Los costos esperados de cada decisión son

$$E[C \mid \text{declarar fake}] = C_{FP}\,(1-p), \qquad E[C \mid \text{declarar real}] = C_{FN}\,p.$$

Conviene declarar la noticia como falsa cuando $C_{FP}(1-p) < C_{FN}\,p$, es decir,

$$\boxed{p > t^* = \frac{C_{FP}}{C_{FP} + C_{FN}}}.$$

Con costos iguales $t^* = 1/2$; si el falso positivo es 10 veces más costoso, $t^* = 10/11$. Esta fórmula es óptima solo si las probabilidades están bien calibradas.

3. Trabajo en clase (en papel)

Resolvimos a mano los ejercicios con un ejemplo de juguete de cuatro palabras (*gratis, premio, hola, mañana*) y dos clases, *spam* y *ham*. El modelo es el mismo que aplicamos después a noticias.

3.1. Ejercicio 1: Bayes ingenuo a mano

Con $P(\text{spam}) = P(\text{ham}) = \frac{1}{2}$ y los conteos de la Tabla 2, calculamos con suavizado de Laplace $P(\text{spam} \mid m)$ para $m = \langle \text{gratis, hola, premio} \rangle$.

Palabra	Spam	Ham	Total
gratis	3	0	3
premio	2	0	2
hola	1	3	4
mañana	0	3	3
Total	6	6	12

Tabla 2: Frecuencia de las palabras por clase en el ejemplo de juguete.

Solución. Con $N(\text{spam}) = N(\text{ham}) = 6$ y $|V| = 4$, el denominador del suavizado es $N(c) + |V| = 10$ en ambas clases, y $P(w \mid c) = \frac{N(w,c)+1}{10}$:

	gratis	premio	hola	mañana
$P(w \mid \text{spam})$	4/10	3/10	2/10	1/10
$P(w \mid \text{ham})$	1/10	1/10	4/10	4/10

Por independencia condicional,

$$P(m \mid \text{spam}) = \tfrac{4}{10} \cdot \tfrac{2}{10} \cdot \tfrac{3}{10} = 0.024,$$
$$P(m \mid \text{ham}) = \tfrac{1}{10} \cdot \tfrac{4}{10} \cdot \tfrac{1}{10} = 0.004.$$

La probabilidad total del mensaje es $P(m) = 0.024 \cdot \frac{1}{2} + 0.004 \cdot \frac{1}{2} = 0.014$, y por la regla de Bayes

$$P(\text{spam} \mid m) = \frac{0.024 \cdot \frac{1}{2}}{0.014} = \frac{6}{7} \approx 0.8571.$$

Resultado: el mensaje es spam con probabilidad ≈ 85.71 %. Observamos que "mañana" tenía conteo cero en spam y, gracias al suavizado, queda con probabilidad 1/10 en lugar de 0.

3.2. Ejercicio 2: Bayes ingenuo es un clasificador lineal

Aplicamos logaritmos al cociente de posteriores (el denominador $P(W)$ se cancela):

$$\log \frac{P(\text{spam} \mid W)}{P(\text{ham} \mid W)} = \log \frac{P(\text{spam})}{P(\text{ham})} + \sum_{j=1}^{|V|} x_j \log \frac{\theta_{j,\text{spam}}}{\theta_{j,\text{ham}}} = \mathbf{w}^T \mathbf{x} + b,$$

con $w_j = \log \frac{\theta_{j,\text{spam}}}{\theta_{j,\text{ham}}}$ y $b = \log \frac{P(\text{spam})}{P(\text{ham})}$. Como las a priori son iguales, $b = \log 1 = 0$. Los pesos, con las probabilidades suavizadas del ejercicio anterior, son:

	gratis	premio	hola	mañana
w_j	$\log 4$	$\log 3$	$\log \frac{1}{2}$	$\log \frac{1}{4}$
valor numérico	1.386	1.099	-0.693	-1.386

Para $\mathbf{x} = [1, 1, 1, 0]^T$ (gratis, premio, hola, mañana):

$$z = \log 4 + \log 3 + \log \tfrac{1}{2} = \log 6 \approx 1.792, \qquad P(\text{spam} \mid W) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + 1/6} = \frac{6}{7} \approx 0.8571,$$

que coincide con el Ejercicio 1.

Comparación con la regresión logística. Ambos modelos tienen la forma $P(\text{spam} \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$. La diferencia está en cómo obtenemos los pesos: en Bayes ingenuo cada w_j se calcula *contando* y depende solo de la palabra j; en regresión logística todos los pesos se ajustan *conjuntamente* minimizando una función de pérdida, por lo que cada peso toma en cuenta lo que ya explican las demás palabras.

3.3. Ejercicio 3: umbral y costos

Si clasificar como spam un mensaje legítimo cuesta 10 veces más que dejar pasar un spam, tenemos $C_{FP} = 10$ y $C_{FN} = 1$. Sea $p = P(\text{spam} \mid m)$:

- Costo esperado de declarar **spam**: solo hay error si el mensaje era legítimo, con costo $C_{FP}\,(1-p) = 10(1-p)$.
- Costo esperado de declarar **legítimo**: solo hay error si era spam, con costo $C_{FN}\,p = p$.

Declaramos spam cuando $10(1-p) < p$:

$$\begin{aligned} 10 - 10p &< p \\ 10 &< 11p \\ \tfrac{10}{11} &< p. \end{aligned}$$

Resultado:

$$\boxed{P(\text{spam} \mid m) > \tfrac{10}{11} \approx 0.9091} \qquad \Longleftrightarrow \qquad \mathbf{w}^T\mathbf{x} + b > \log 10 \approx 2.303.$$

Para el mensaje del Ejercicio 1, $P(\text{spam} \mid m) = 6/7 \approx 0.8571 < 10/11$, por lo que **no** lo marcamos como spam: aunque con umbral 0.5 sí lo marcaríamos, con este costo asimétrico no hay certeza suficiente para arriesgarnos a bloquear un mensaje legítimo. El caso general $t^* = C_{FP}/(C_{FP} + C_{FN})$ lo deducimos en la Sección 2.8.

3.4. Ejercicio 4: métricas en un problema desbalanceado

Construimos matrices de confusión para 1,000 mensajes, de los cuales 130 son spam (clase positiva).

	Predicción: spam	Predicción: ham
Real: spam	0	130
Real: ham	0	870

Tabla 3: Matriz de confusión del Clasificador 1 ($VP = 0$, $FN = 130$, $FP = 0$, $VN = 870$).

Clasificador 1: declara todos los mensajes legítimos. Exactitud $= \frac{0+870}{1000} = 0.87$. La precisión es $\frac{0}{0+0}$, indefinida (la tomamos como 0 por convención). Exhaustividad $= \frac{0}{130} = 0$ y $F_1 = 0$.

	Predicción: spam	Predicción: ham
Real: spam	110	20
Real: ham	15	855

Tabla 4: Matriz de confusión del Clasificador 2 ($VP = 110$, $FN = 20$, $FP = 15$, $VN = 855$).

Clasificador 2: valores plausibles elegidos por el equipo.

$$\text{Exactitud} = \frac{110 + 855}{1000} = 0.965, \qquad \text{Precisión} = \frac{110}{125} = 0.88,$$

$$\text{Exhaustividad} = \frac{110}{130} \approx 0.8462, \qquad F_1 = \frac{2 \cdot 110}{2 \cdot 110 + 15 + 20} = \frac{220}{255} \approx 0.8627.$$

	Exactitud	Precisión	Exhaustividad	F_1
Clasificador 1 (todo legítimo)	0.870	indef. (0)	0.000	0.000
Clasificador 2 (plausible)	0.965	0.880	0.846	0.863

Tabla 5: Resumen de métricas de los dos clasificadores hipotéticos.

¿Por qué la exactitud es engañosa? El Clasificador 1 no detecta ninguno de los 130 spam y aun así alcanza 87 % de exactitud, solo porque 87 % de los mensajes es legítimo. Con clases desbalanceadas, predecir siempre la clase mayoritaria ya produce una exactitud alta, por lo que debemos compararla contra ese piso ($1 - 0.13 = 0.87$) y evaluar el desempeño sobre la clase de interés con precisión, exhaustividad y F_1.

4. Etapa 1: análisis, representación y *baseline*

4.1. Exploración del corpus

Desbalance y duplicados. Eliminamos las noticias con texto igual y el corpus pasa de 44,898 a **38,646** noticias (Tabla 6). La eliminación afecta sobre todo a la clase falsa: una cuarta parte de sus noticias estaba duplicada, contra 1.1 % de las verdaderas. Como consecuencia, la clase mayoritaria cambia de *Fake* (52.3 %) a *Real* (54.8 %). Eliminamos los duplicados antes de dividir los datos para evitar que una misma noticia quede en entrenamiento y en prueba.

Clase	Antes	Duplicados eliminados	Después	Proporción
Fake	23,481	6,026	17,455	45.2 %
Real	21,417	226	21,191	54.8 %
Total	44,898	6,252	38,646	100 %

Tabla 6: Efecto de la eliminación de duplicados sobre el tamaño de cada clase.

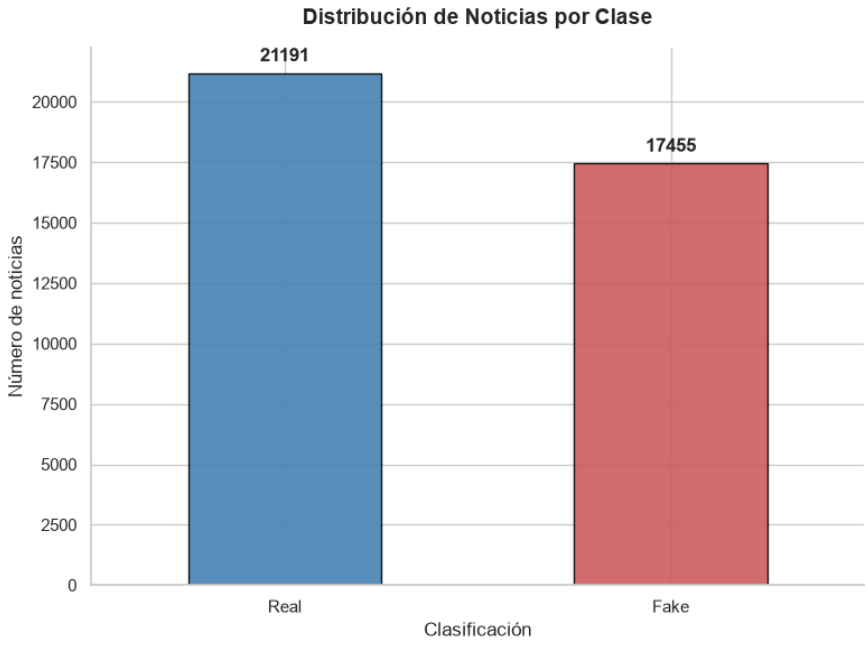


Figura 1: Distribución de clases después de eliminar duplicados.

Longitud. La Tabla 7 resume la longitud de las noticias. Las medianas son casi idénticas (2,235 contra 2,216 caracteres; 376 contra 359 palabras); la diferencia en las medias se debe a que hay pocos textos muy largos en la clase falsa (máximo de 51,794 caracteres). Las distribuciones se traslapan (Figura 2), por lo que la longitud por sí sola no distingue bien las clases.

	Clase	Media	Desv. est.	Mínimo	Mediana	Máximo
Caracteres	Fake	2,549.8	2,201.3	1	2,235	51,794
	Real	2,378.5	1,683.8	152	2,216	29,781
Palabras	Fake	425.3	355.2	0	376	8,135
	Real	384.8	273.8	22	359	5,172

Tabla 7: Longitud de las noticias por clase (después de eliminar duplicados).

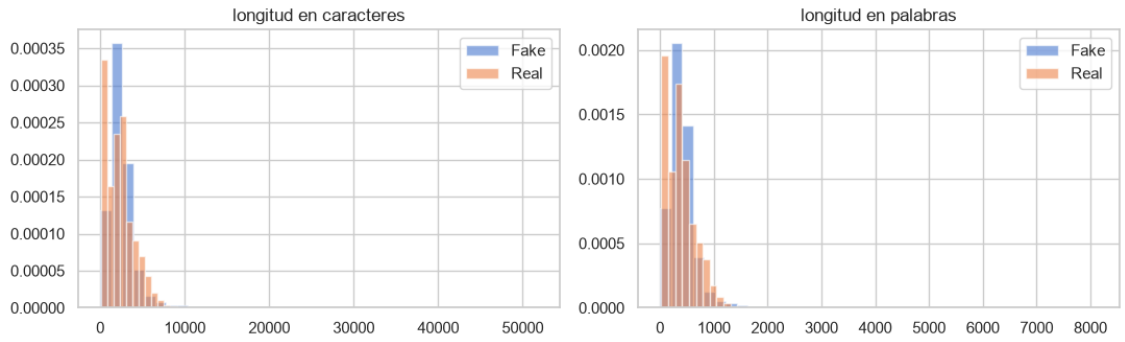


Figura 2: Histogramas de longitud en caracteres y en palabras, por clase.

Señalamos que el mínimo de palabras en la clase falsa es 0: existen noticias con texto vacío o casi vacío (363 noticias con menos de 20 palabras, todas ellas falsas). Las conservamos en el corpus y lo discutimos en las limitaciones.

Vocabulario característico. El vocabulario del corpus completo tiene 122,341 tipos. Las palabras más frecuentes en ambas clases son palabras funcionales (*the, to, of, and, a, in*); entre las diferencias, *said* aparece entre las más frecuentes solo en la clase verdadera (Figura 3).

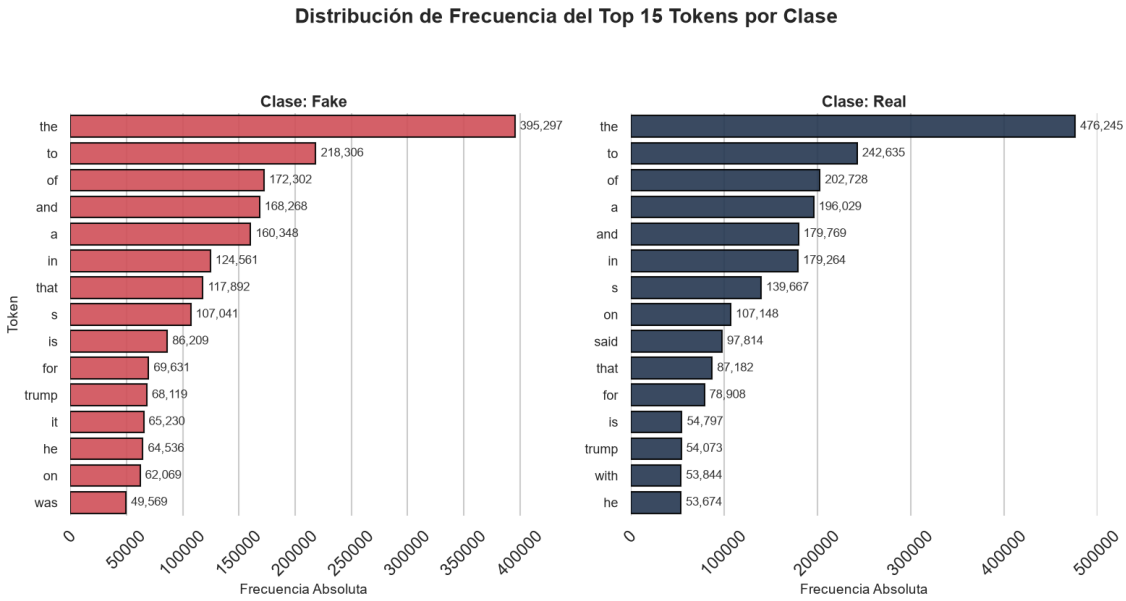


Figura 3: Los 15 tokens más frecuentes en noticias falsas y verdaderas.

Ley de Zipf. La frecuencia de las palabras en función de su rango sigue una ley de potencias [7], muy parecida en ambas clases (Figura 4): la curva real queda por encima de la recta $1/r$ en la parte media y cae por debajo en la cola. Pensamos que esto ocurre porque la mayoría de los tipos del vocabulario son raros. De igual forma, las diferencias entre clases no estarán en la forma de la distribución de frecuencias, sino en *qué* palabras se usan.

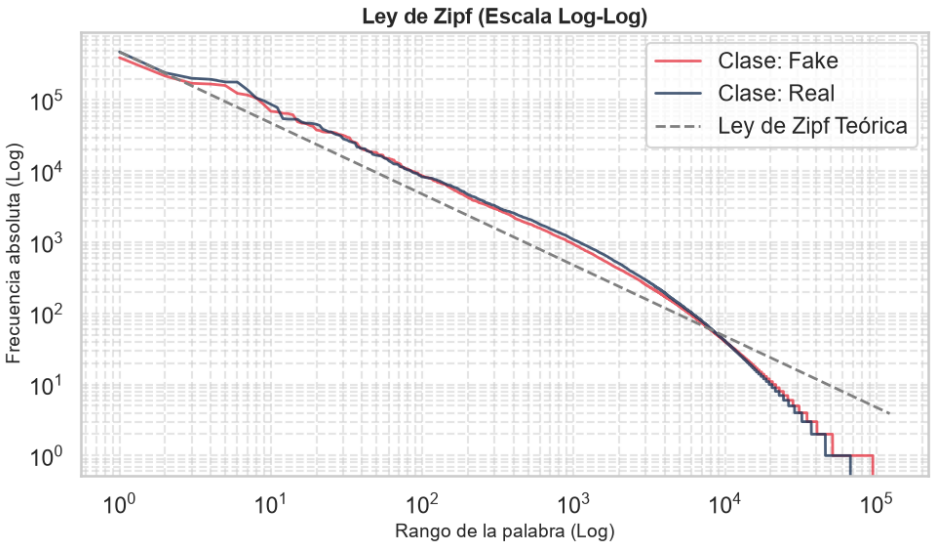


Figura 4: Ley de Zipf en escala log-log por clase, con la recta de referencia $1/r$.

Marcas de la fuente. Medimos la frecuencia de dos marcas en el texto: la aparición de la palabra "reuters" y de "http".

Noticias que contienen	Fake	Real
la palabra "reuters"	1.26 %	99.83 %
"http" (URL)	14.41 %	≈ 0 % (una noticia)

Tabla 8: Presencia de marcas de la fuente en el texto, por clase.

Hallazgo

El 99.83 % de las noticias verdaderas contiene "Reuters" (contra 1.26 % de las falsas) y las URLs aparecen casi solo en las falsas. En la Sección 7 medimos cuánto depende el desempeño de estas marcas.

4.2. Partición de los datos

Dividimos los datos de forma **estratificada** en 70 % entrenamiento (*Train*), 15 % validación (*Dev*) y 15 % prueba (*Test*), con semilla aleatoria fija para reproducibilidad (Tabla 9). Train ajusta el modelo, Dev nos sirve para elegir hiperparámetros y umbrales, y usamos Test solo para reportar el desempeño final. La estratificación conserva ≈ 45.2 % de noticias falsas en las tres particiones.

Partición	Real	Fake	Total	Proporción Fake
Train	14,834	12,218	27,052	0.4516
Dev	3,178	2,619	5,797	0.4518
Test	3,179	2,618	5,797	0.4516

Tabla 9: Particiones estratificadas del corpus.

4.3. Representación: bolsa de palabras

Implementamos la representación con `CountVectorizer` de `scikit-learn` [6], con minúsculas y un patrón de tokens propio (`[a-z0-9£$]+|!`) que conserva números y signos que pueden ser informativos. Aprendemos el vocabulario **solo con Train**: ignoramos las palabras de Dev y Test ausentes en Train, para no filtrar información del conjunto de prueba. Obtuvimos:

- una matriz de entrenamiento de 27,052 × 101,945 (documentos × términos);
- una densidad de 0.21 % de entradas distintas de cero, por lo que la guardamos como matriz dispersa (`scipy.sparse`) y no la convertimos a densa.

Traslape del vocabulario. Considerando los términos con al menos 5 apariciones en la clase, 8,740 aparecen solo en noticias falsas, 7,075 solo en verdaderas y 17,509 en ambas. La similitud coseno entre los perfiles de frecuencia de ambas clases es 0.972: las clases comparten gran parte del vocabulario, pero cada una tiene un conjunto propio considerable (Figura 5).

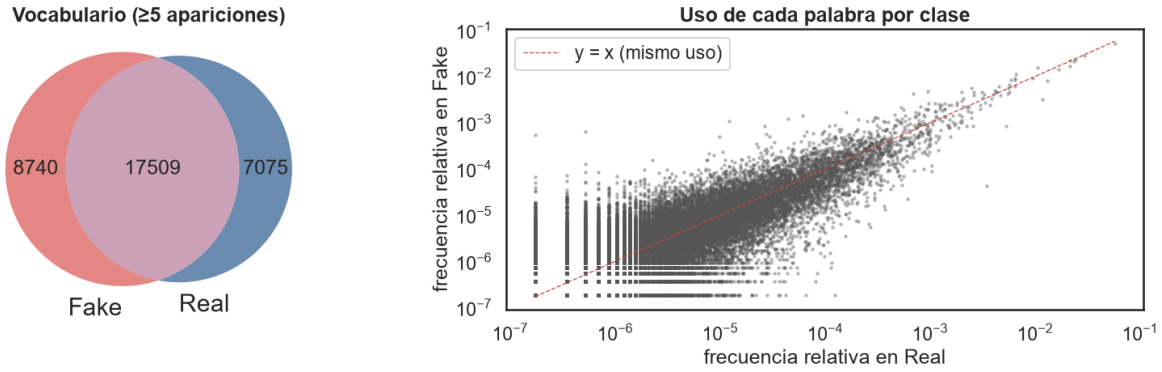


Figura 5: Traslape de vocabulario entre noticias falsas y verdaderas.

4.4. *Baseline*: Bayes ingenuo multinomial

Nuestro *baseline* es Bayes ingenuo multinomial sobre BoW (Sección 2.3). Estimamos las probabilidades a priori con las proporciones de Train. El único hiperparámetro es el suavizado α, que elegimos en Dev entre $\{0.01, 0.1, 0.5, 1, 2\}$ según el F_1 de la clase falsa (Tabla 10, Figura 6). Obtuvimos $\alpha = 0.1$. Los F_1 en Dev van de 0.9492 a 0.9510, una diferencia de pocas noticias, por lo que el modelo es poco sensible a α en este rango.

α	Exactitud	Precisión	Exhaustividad	F_1	AUC-ROC
0.01	0.9551	0.9436	0.9580	0.9507	0.9777
0.10	**0.9555**	**0.9453**	**0.9569**	**0.9510**	**0.9780**
0.50	0.9543	0.9452	0.9542	0.9496	0.9780
1.00	0.9539	0.9448	0.9538	0.9493	0.9781
2.00	0.9539	0.9451	0.9534	0.9492	0.9783

Tabla 10: Selección del suavizado α de Bayes ingenuo en Dev.

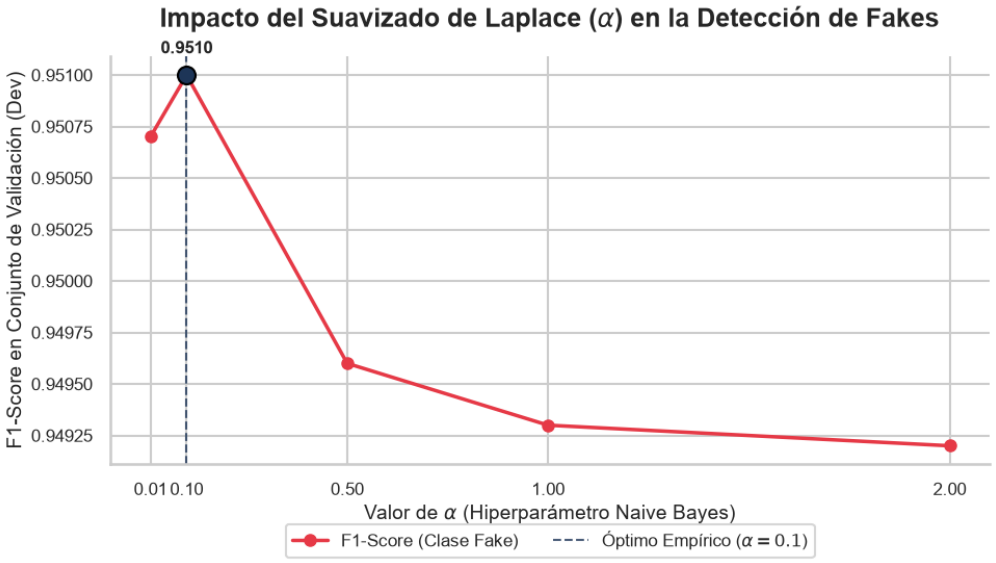


Figura 6: Efecto del suavizado α sobre las métricas de Bayes ingenuo en Dev.

Resultados en Test. Con $\alpha = 0.1$ y umbral 0.5, el *baseline* obtiene exactitud 0.9517, precisión 0.9441, exhaustividad 0.9492, $F_1 = 0.9467$ y AUC-ROC 0.9760 (Tabla 11). Supera por mucho el piso de exactitud de 0.5484. La matriz de confusión (Figura 7) muestra 147 noticias reales marcadas como falsas (2.54 % de las 5,797 evaluadas) y 133 falsas que pasaron (2.29 %).

	Exactitud	Precisión	Exhaustividad	F_1	AUC-ROC
Bayes ingenuo (BoW)	0.9517	0.9441	0.9492	0.9467	0.9760

Tabla 11: Desempeño del *baseline* en Test (umbral 0.5).

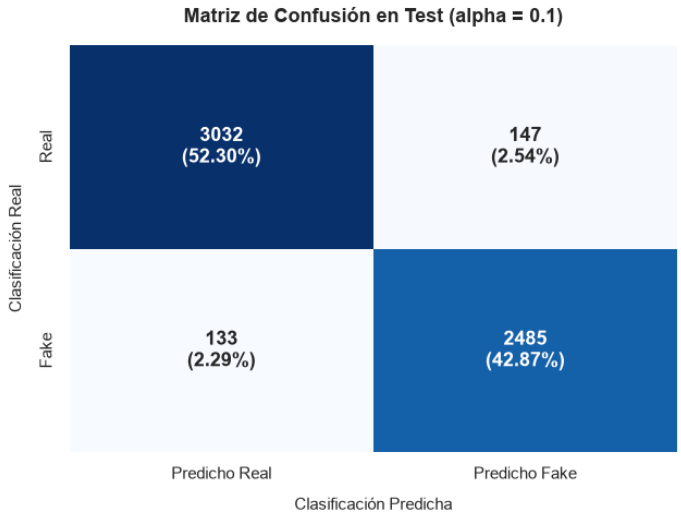


Figura 7: Matriz de confusión del *baseline* en Test.

Validación de la implementación. Implementamos también Bayes ingenuo manualmente (conteos, suavizado y logaritmos) y lo comparamos con `scikit-learn`. Las predicciones son idénticas, la exactitud, precisión, exhaustividad y F_1 coinciden exactamente, y las log-probabilidades $\log P(w \mid c)$ no difieren.

5. Etapa 2: TF-IDF y regresión logística

5.1. Representación TF-IDF

Nuestra segunda representación es TF-IDF (Sección 2.2), con `TfidfVectorizer` ajustado solo con Train y la siguiente configuración: minúsculas, eliminación de *stopwords* en inglés, `min_df=5` (ignoramos términos presentes en menos de 5 documentos), `max_df=0.8` (ignoramos términos presentes en más del 80 % de los documentos), unigramas, *tf* sublineal e *idf* suavizado. El vocabulario resultante es de **31,213** términos, frente a 101,945 de BoW.

5.2. Regresión logística con regularización L_2

Entrenamos la regresión logística con el solucionador `liblinear` sobre la matriz TF-IDF. Elegimos el parámetro C en Dev según el F_1 (Tabla 12, Figura 8). Con regularización fuerte ($C = 0.01$) el modelo sacrifica exhaustividad (0.8545); al relajarla, todas las métricas mejoran. El mejor valor es $C = 100$.

C	Exactitud	Precisión	Exhaustividad	F_1	AUC-ROC
0.01	0.9284	0.9850	0.8545	0.9152	0.9903
0.10	0.9729	0.9809	0.9588	0.9697	0.9958
1.00	0.9865	0.9904	0.9798	0.9850	0.9985
10.0	0.9907	0.9938	0.9855	0.9896	0.9992
100	**0.9919**	**0.9946**	**0.9874**	**0.9910**	**0.9994**

Tabla 12: Selección de C en Dev para la regresión logística con TF-IDF.

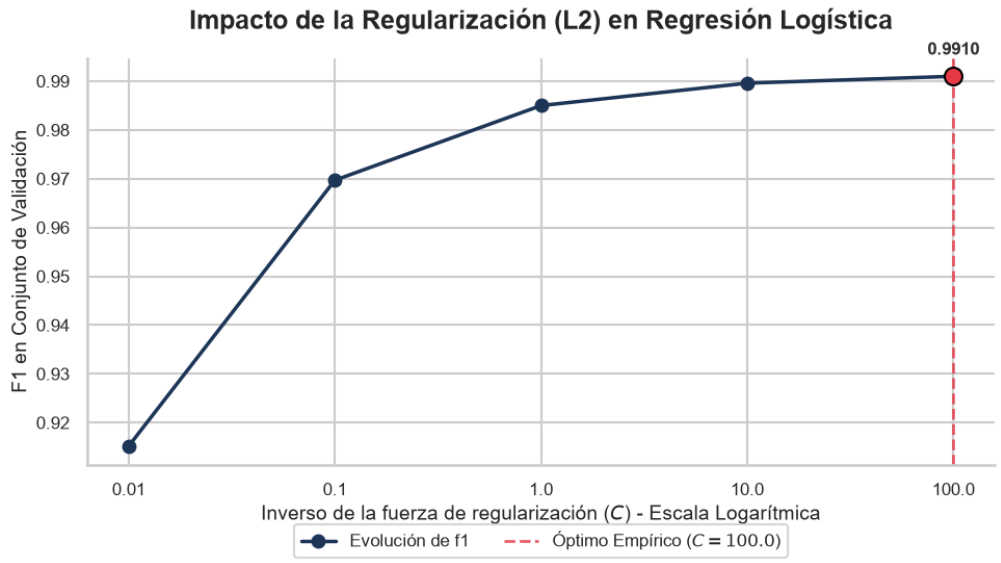


Figura 8: Efecto de la regularización L_2 (C) sobre el F_1 en Dev.

5.3. Comparación de los modelos en Test

Con $C = 100$ y umbral 0.5, la regresión logística supera al *baseline* en todas las métricas (Tabla 13, Figura 9).

	Exactitud	Precisión	Exhaustividad	F_1	AUC-ROC
Bayes ingenuo (BoW)	0.9517	0.9441	0.9492	0.9467	0.9760
Regresión logística (TF-IDF)	0.9938	0.9950	0.9912	0.9931	0.9995

Tabla 13: Comparación de ambos modelos en Test (umbral 0.5).

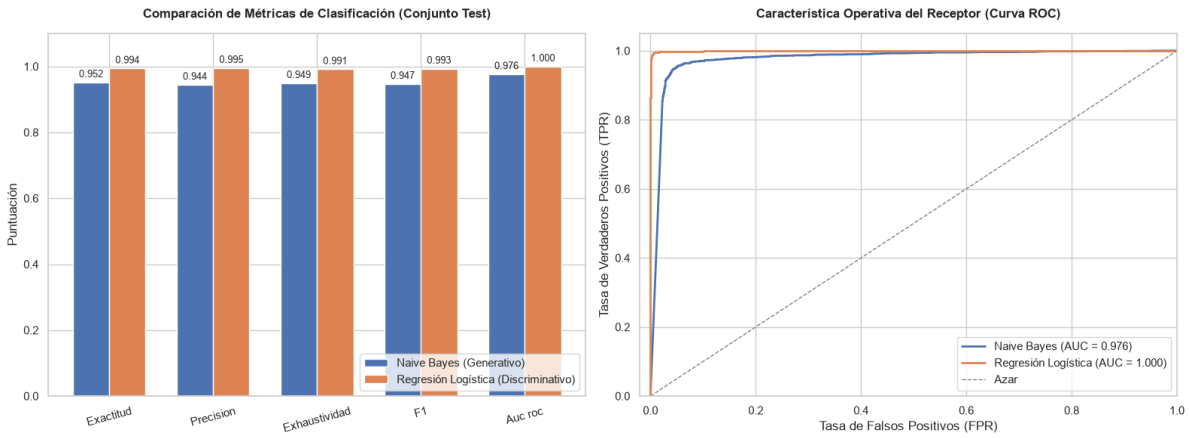


Figura 9: Métricas en Test de ambos modelos y curvas ROC.

El F_1 pasa de 0.9467 a 0.9931 y el AUC-ROC de 0.9760 a 0.9995. En número de errores sobre las 5,797 noticias de Test, Bayes ingenuo comete 280 (147 falsos positivos y 133 falsos negativos) y la regresión logística 36, una reducción de ≈ 87 %. Pensamos que esto es coherente con la discusión teórica: Bayes ingenuo sobreestima la evidencia de términos correlacionados, mientras que la regresión logística ajusta los pesos conjuntamente.

5.4. Ablación: ¿efecto del modelo o de la representación?

Las configuraciones oficiales difieren en dos cosas a la vez (representación y modelo). Para separar los efectos, comparamos las cuatro combinaciones (BoW o TF-IDF, con cada modelo) usando **los mismos parámetros** de tokenización y filtro de vocabulario (`min_df=5`, mismo patrón de tokens y sin eliminar *stopwords*), así solo cambia el ponderado (conteos o TF-IDF). Mostramos los resultados en Dev en la Tabla 14.

Representación	Modelo	Exactitud	Precisión	Exhaust.	F_1	AUC
BoW	Bayes ingenuo	0.9546	0.9425	0.9580	0.9502	0.9776
BoW	Regresión log.	0.9934	0.9924	0.9931	0.9927	0.9984
TF-IDF	Bayes ingenuo	0.9470	0.9450	0.9374	0.9412	0.9866
TF-IDF	Regresión log.	0.9941	0.9965	0.9905	0.9935	0.9996

Tabla 14: Ablación representación $\times$ modelo en Dev (vocabulario de 31,680 términos en ambas representaciones).

Lectura de la ablación

- **La mejora proviene principalmente del modelo, no de la representación.** Con la misma representación, la regresión logística supera a Bayes ingenuo por 4.3 puntos de F_1 con BoW (0.9927 contra 0.9502) y por 5.2 puntos con TF-IDF (0.9935 contra 0.9412).

- Pasar de BoW a TF-IDF modifica poco el F_1 de la regresión logística (0.9927 a 0.9935) y lo *empeora* en Bayes ingenuo (0.9502 a 0.9412).

5.5. Efecto del umbral de decisión

Los dos modelos entregan una probabilidad de que la noticia sea falsa, y por defecto declaramos "falsa" si es ≥ 0.5. Estudiamos el efecto de cambiar ese umbral en Dev (Figura 10; Tabla 15).

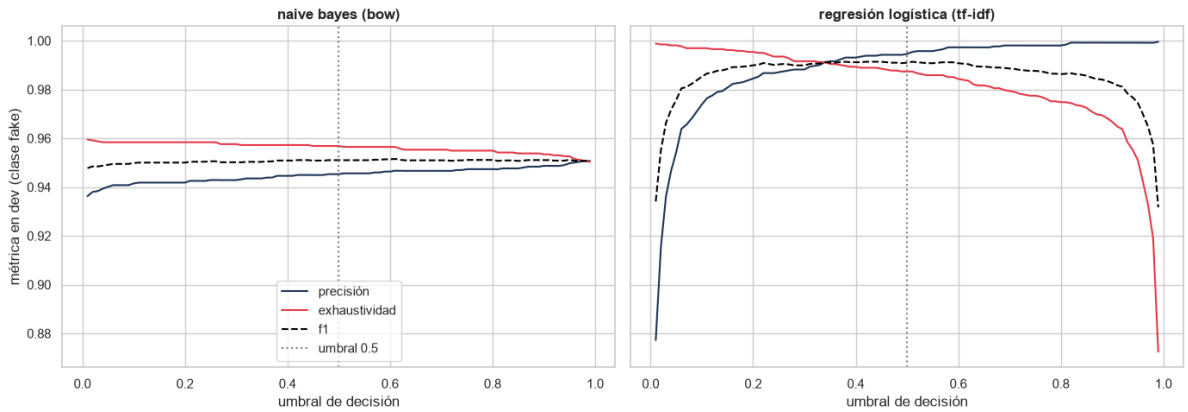


Figura 10: Precisión, exhaustividad y F_1 en Dev al variar el umbral de decisión, para cada modelo.

Bayes ingenuo. El umbral casi no tiene efecto. El 98.86 % de las noticias de Dev recibe una probabilidad menor que 0.01 o mayor que 0.99: el modelo casi nunca duda. Al mover el umbral de 0.1 a 0.9 el total de errores solo pasa de 265 a 257.

Regresión logística. Solo 87.99 % de las noticias queda saturado (12.01 % tiene probabilidad entre 0.01 y 0.99) y el umbral *sí* controla el intercambio entre errores: subirlo reduce los falsos positivos a costa de más falsos negativos, y bajarlo hace lo contrario. Alrededor de 0.5 el F_1 presenta una meseta amplia (Figura 10); el mejor F_1 en Dev (0.9914) se obtiene con umbral 0.38, solo 0.0004 sobre el de 0.5.

	Bayes ingenuo (BoW)				Regresión logística (TF-IDF)			
Umbral	FP	FN	Prec.	Exhaust.	FP	FN	Prec.	Exhaust.
0.1	156	109	0.9415	0.9584	70	8	0.9739	0.9969
0.3	152	111	0.9429	0.9576	31	22	0.9882	0.9916
0.5	145	113	0.9453	0.9569	14	33	0.9946	0.9874
0.7	141	117	0.9467	0.9553	5	54	0.9981	0.9794
0.9	135	122	0.9487	0.9534	2	87	0.9992	0.9668

Tabla 15: Errores en Dev a distintos umbrales. FP: noticias reales marcadas como falsas; FN: noticias falsas que pasaron.

En la regresión logística, pasar de 0.5 a 0.9 evita 12 falsos positivos (14 a 2) pero deja pasar 54 falsas más (33 a 87); pasar de 0.5 a 0.1 atrapa 25 falsas más (33 a 8) pero marca como falsas a 56 reales más (14 a 70). Con costos iguales el umbral 0.5 ya es casi óptimo (47 errores en total, el mínimo de la tabla).

6. Etapa 3: atributos adicionales y ajuste del umbral

6.1. Atributos de forma del texto

Hasta ahora los modelos solo ven qué palabras hay. Añadimos cuatro atributos que describen la *forma* del texto, calculados sobre el texto original (necesario porque las mayúsculas se pierden al pasar a minúsculas):

- `log_longitud`: $\log(1 + \text{número de caracteres})$;
- `log_digitos`: $\log(1 + \text{número de dígitos})$;
- `log_urls`: $\log(1 + \text{número de URLs})$;
- `prop_mayusculas`: mayúsculas entre letras totales.

Pasamos los conteos por $\log(1 + \cdot)$ porque hay textos de más de 50,000 caracteres y unos pocos valores extremos dominarían al modelo. Después los escalamos a $[0, 1]$ con un escalador ajustado **solo con Train** (con recorte de valores fuera de rango en Dev y Test) y los concatenamos a la matriz TF-IDF (31,213 $\rightarrow$ 31,217 columnas). El escalado es necesario porque la regularización L_2 es sensible a la escala de las variables y TF-IDF ya vive en $[0, 1]$.

Los atributos por sí solos separan poco las clases. La Tabla 16 muestra el promedio de cada atributo por clase y su AUC-ROC al usarlo solo como puntaje (0.5 equivale al azar). Todos están cerca de 0.5. El único con diferencia clara es el número de URLs, pero la mayoría de las noticias falsas (86 %) no tiene URLs.

Atributo	Promedio Fake	Promedio Real	AUC solo
`log_longitud`	7.570	7.480	0.525
`log_digitos`	2.125	2.087	0.499
`log_urls`	0.132	0.002	0.571
`prop_mayusculas`	0.057	0.052	0.466

Tabla 16: Atributos de forma: promedio por clase y AUC-ROC de cada uno por separado (Train).

Efecto sobre el modelo. En Dev (Tabla 17), un modelo que usa *solo* los atributos apenas supera al azar (exactitud 0.6372, AUC 0.6363). Al combinarlos con TF-IDF el F_1 pasa de 0.9910 a 0.9912, es decir, una sola noticia de diferencia.

Entrada	Exactitud	Precisión	Exhaust.	F_1	AUC
TF-IDF	0.9919	0.9946	0.9874	0.9910	0.9994
TF-IDF + atributos	0.9921	0.9950	0.9874	0.9912	0.9994
Solo atributos	0.6372	0.8554	0.2371	0.3713	0.6363

Tabla 17: Aporte de los atributos en Dev (regresión logística).

En Test (Tabla 18, Figura 11) las métricas son prácticamente idénticas: las diferencias entre el modelo con y sin atributos son $+0.0008$ en precisión, -0.0007 en exhaustividad, 0.0000 en F_1 y -0.0001 en AUC-ROC.

	Exactitud	Precisión	Exhaust.	F_1	AUC
Bayes ingenuo (BoW)	0.9517	0.9441	0.9492	0.9467	0.9760
Reg. logística (TF-IDF)	0.9938	0.9950	0.9912	0.9931	0.9995
Reg. logística (TF-IDF + atributos)	0.9938	0.9958	0.9905	0.9931	0.9995

Tabla 18: Comparación en Test, umbral 0.5.

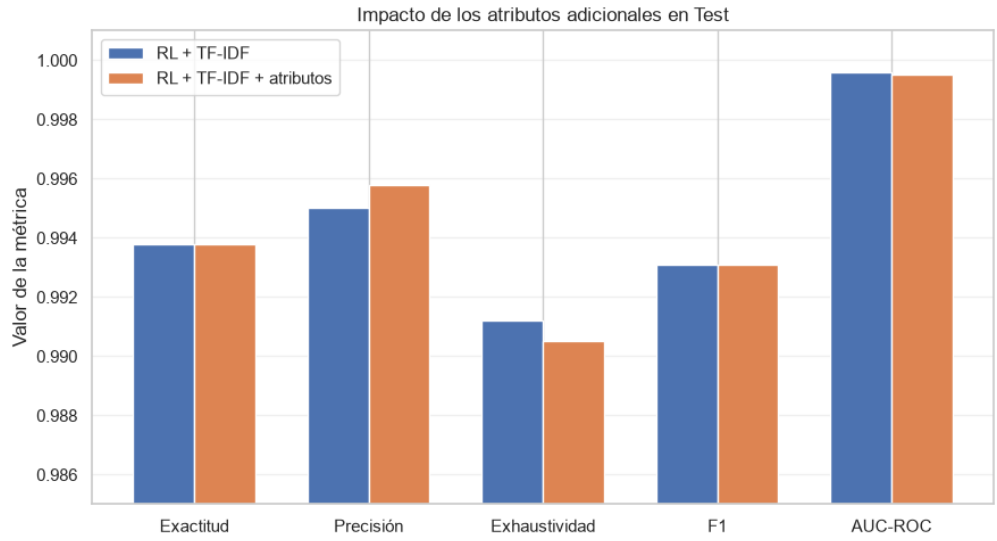


Figura 11: Diferencia en Test entre la regresión logística con atributos y sin ellos.

Los pesos que aprendimos para los atributos (escalados a $[0, 1]$, por lo que son comparables entre sí) son $+8.13$ para `log_urls`, $+4.76$ para `prop_mayusculas`, -1.11 para `log_digitos` y -2.67 para `log_longitud`. Un peso grande no implica un impacto grande: las URLs tienen el mayor peso, pero solo son distintas de cero en una minoría de las noticias falsas.

Resultado de la mejora

Los atributos adicionales **no mejoran** el desempeño. Una explicación plausible es que la información más útil que aportan, la presencia de URLs, ya está contenida en las palabras (por ejemplo, el token *https* es el de mayor log-razón hacia la clase falsa) y que el modelo basado en TF-IDF ya está cerca de su tope. Además, el atributo de URLs refleja una marca de la fuente (Tabla 8) y no una propiedad de la veracidad.

6.2. Ajuste del umbral

Criterio. Para cada modelo elegimos en **Dev** el umbral que maximiza el F_1 de la clase falsa (búsqueda sobre $0.01, 0.02, \ldots, 0.99$) y lo aplicamos después, sin reajustarlo, sobre Test. Este criterio trata ambos tipos de error con un costo comparable, que equivale al caso $C_{FP} = C_{FN}$ de la Sección 2.8.

Modelo	Umbral (Dev)	F_1 Dev	Prec. Dev	Exhaust. Dev
Bayes ingenuo (BoW)	0.61	0.9516	0.9467	0.9565
Reg. logística (TF-IDF)	0.38	0.9914	0.9931	0.9897
Reg. logística (TF-IDF + atributos)	0.40	0.9916	0.9935	0.9897

Tabla 19: Umbral que maximiza el F_1 en Dev para cada modelo.

Modelo	Umbral	Exact.	Prec.	Exhaust.	F_1	AUC
Bayes ingenuo (BoW)	0.61	0.9514	0.9444	0.9481	0.9462	0.9760
Reg. logística (TF-IDF)	0.38	0.9938	0.9939	0.9924	0.9931	0.9995
Reg. logística (TF-IDF + atributos)	0.40	0.9938	0.9939	0.9924	0.9931	0.9995

Tabla 20: Desempeño en Test con el umbral ajustado en Dev.

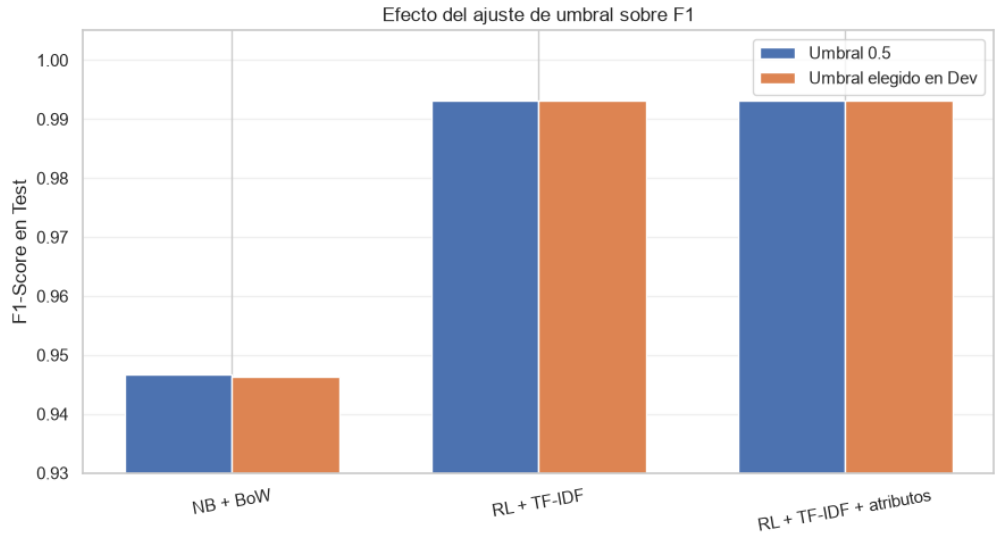
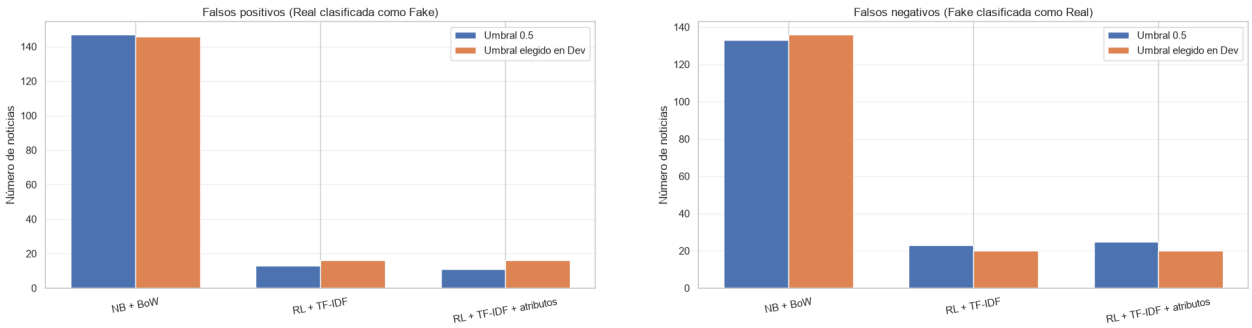


Figura 12: F_1 en Test con umbral 0.5 frente al umbral ajustado en Dev.



(a) Falsos positivos

(b) Falsos negativos

Figura 13: Errores en Test con umbral 0.5 frente al umbral ajustado en Dev.

Resultados en Test con el umbral heredado de Dev. El ajuste del umbral **no produce una mejora real** en Test. En Bayes ingenuo el F_1 baja ligeramente (0.9467 a 0.9462). En la regresión logística el F_1 queda en 0.9931: pasar de 0.5 a 0.38 cambia de lugar tres errores (de 13 a 16 falsos positivos y de 23 a 20 falsos negativos).

Relación con el análisis de costos. El umbral cambia de lugar solo cuando un error es claramente más costoso que el otro. Para ilustrarlo, en la Tabla 21 calculamos el costo total en Dev de la regresión logística (TF-IDF) como $r \cdot FP + FN$ para distintas razones $r = C_{FP}/C_{FN}$, a partir de los errores de la Tabla 15. Son valores *ilustrativos*, no una elección de nuestro equipo. A medida que r crece, el umbral de menor costo se desplaza hacia la derecha (de 0.5 a 0.7), en el sentido que predice $t^* = r/(1+r)$ (por ejemplo, 0.833 para $r = 5$ y 0.909 para $r = 10$). Dentro de los umbrales de la tabla, el mínimo para $r = 10$ se obtiene en 0.7 y no en 0.9, lo que sugiere que las probabilidades de la regresión logística no están perfectamente calibradas.

Razón $r = C_{FP}/C_{FN}$	$t = 0.1$	$t = 0.3$	$t = 0.5$	$t = 0.7$	$t = 0.9$
1	78	53	**47**	59	89
2	148	84	**61**	64	91
5	358	177	103	**79**	97
10	708	332	173	**104**	107

Tabla 21: Costo total ilustrativo $r \cdot FP + FN$ en Dev (regresión logística, TF-IDF). En negritas, el menor de cada fila.

7. Experimentos de control e interpretación

Los experimentos de control verifican si los resultados son razonables y si los modelos aprendieron asociaciones útiles y no artefactos.

7.1. Control 1: palabras con mayor log-razón

Calculamos $w_j = \log P(w_j \mid \text{fake}) - \log P(w_j \mid \text{real})$ con el modelo *baseline*, restringiéndonos a palabras con al menos 5 apariciones en Train (las más raras producen valores extremos por azar). La Tabla 22 muestra las diez palabras más asociadas a cada clase y la Figura 14 las representa.

Más asociadas a *Fake*			Más asociadas a *Real*		
Palabra	Log-razón	Frec.	Palabra	Log-razón	Frec.
https	10.181	2,435	myanmar	−9.765	1,888
21wire	9.009	754	rohingya	−9.489	1,432
js	8.840	637	rakhine	−8.559	565
quot	8.750	582	zuma	−8.351	459
flickr	8.395	408	puigdemont	−8.336	452
somodevilla	8.189	332	fdp	−8.136	370
2017the	8.161	323	suu	−8.054	341
fjs	8.136	315	kyi	−8.054	341
getty	7.946	2,867	anc	−7.933	302
var	7.864	240	mnangagwa	−7.871	284

Tabla 22: Palabras con mayor log-razón por clase (frecuencia en Train ≥ 5).

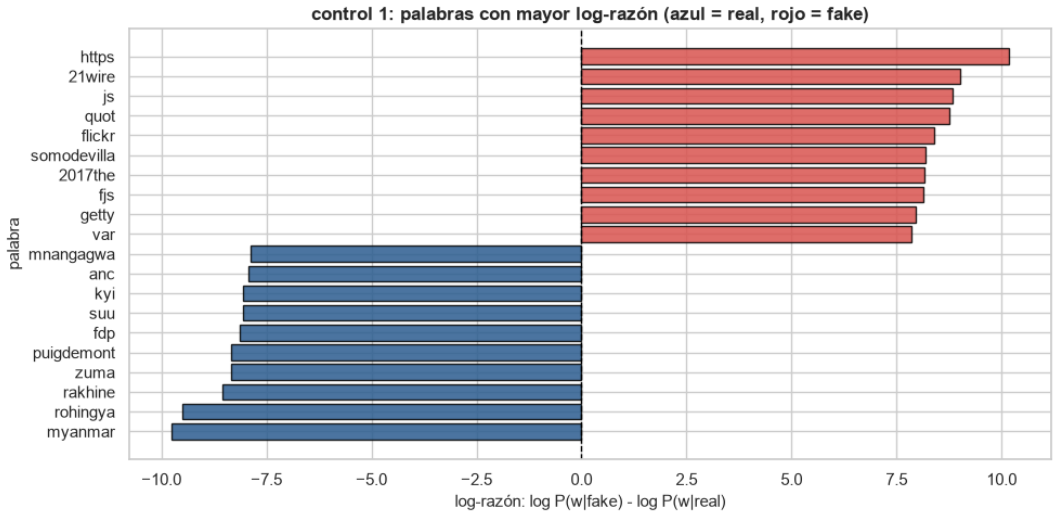


Figura 14: Palabras con mayor log-razón (rojo: asociadas a falsas; azul: asociadas a verdaderas).

Interpretación. Las palabras **no** son interpretables como indicadores de falsedad sino de *fuente, formato y tema*:

- En la clase falsa dominan restos de páginas web y de código (*https*, *js*, *fjs*, *var*, *quot*),

nombres de sitios o de agencias de fotografía (*21wire*, *flickr*, *getty*) y errores de pegado (*2017the*).

- En la clase verdadera dominan nombres propios de eventos de la época: Myanmar y los rohinyás, el *ANC* sudafricano, Puigdemont, Zuma, Mnangagwa, Rajoy. Son términos *temáticos*, ligados a los hechos cubiertos por las noticias verdaderas, no a la veracidad en sí. El token *tmsnrt* (log-razón -7.720, 244 apariciones) probablemente forma parte de enlaces de una agencia de noticias.

El control cumple su función de diagnóstico, pero no muestra que el modelo distinga veracidad: muestra que aprende rasgos de fuente y tema.

7.2. Control 2: etiquetas permutadas

Mezclamos al azar las etiquetas de Train (30 permutaciones independientes), conservando los mismos documentos y el mismo número de etiquetas por clase, reentrenamos Bayes ingenuo y lo evaluamos en Test con las etiquetas verdaderas. Si el modelo aprende señal real, su desempeño debe caer al nivel del azar.

Métrica	Etiquetas reales	Permutadas (media)	Permutadas (desv. est.)
Exactitud	0.9517	0.5108	0.0539
Precisión	0.9441	0.4573	0.0616
Exhaustividad	0.9492	0.4463	0.0637
F_1	0.9467	0.4515	0.0619
AUC-ROC	0.9760	0.5040	0.0641

Tabla 23: Desempeño de Bayes ingenuo en Test con etiquetas reales y con 30 permutaciones.

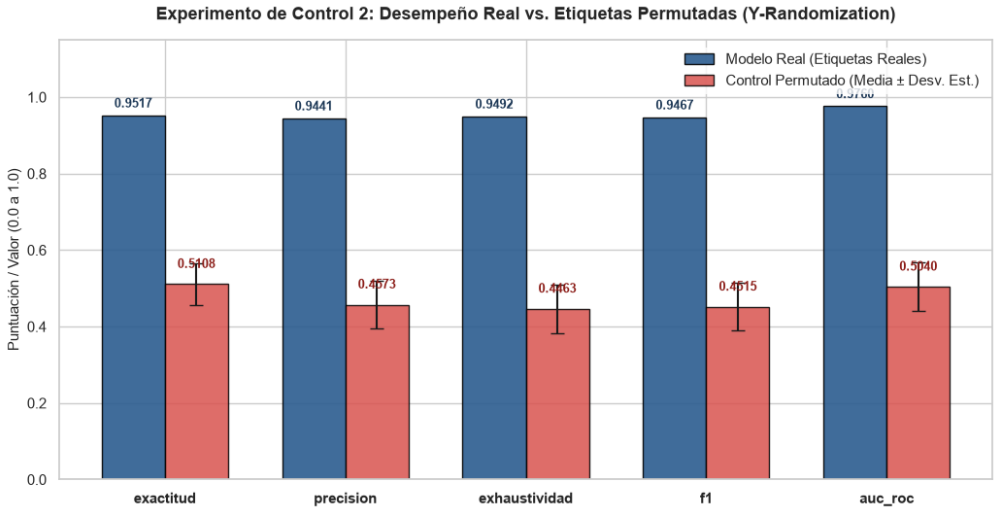


Figura 15: Desempeño con etiquetas reales frente a etiquetas permutadas.

El AUC-ROC cae a 0.504 ± 0.064, es decir, al nivel del azar, y la exactitud a 0.51. Esto descarta errores del procedimiento que inflarían el desempeño (como fugas de información entre Train y Test) y confirma que el modelo aprende relaciones reales entre palabras y etiquetas. Como el corpus está casi balanceado, el efecto de la longitud del documento sobre el puntaje es probablemente menor que en problemas muy desbalanceados.

7.3. Control 3: dependencia de las marcas de fuente

La exploración (Tabla 8) mostró que una regla trivial es muy predictiva: declarar *Real* si el texto contiene "reuters" y *Fake* en caso contrario. Con los porcentajes de la Tabla 8 estimamos, de forma aproximada, que esta regla de una sola palabra alcanza una exactitud y un F_1 de ≈ 0.993 sobre el corpus completo (estimación hecha a partir de porcentajes redondeados, no calculada directamente sobre las predicciones). Es un valor comparable al de la regresión logística (exactitud 0.9938 y F_1 0.9931 en Test), y superior al de Bayes ingenuo.

Para medir cuánto dependen los modelos de estas marcas, eliminamos progresivamente tres elementos del texto: la palabra "Reuters", las URLs y el encabezado de agencia ("CIUDAD (Reuters) -") junto con los créditos finales ("Reporting by…"). Mantuvimos el mismo particionado y solo limpiamos el texto. Los resultados en Dev (Tabla 24) usan Bayes ingenuo con BoW y regresión logística con TF-IDF, con los mismos parámetros de tokenización.

	Bayes ingenuo + BoW			Reg. logística + TF-IDF		
Limpieza (acumulativa)	Exact.	F_1	AUC	Exact.	F_1	AUC
Texto original	0.9536	0.9491	0.9775	0.9941	0.9935	0.9996
− palabra "reuters"	0.9501	0.9455	0.9746	0.9900	0.9889	0.9990
− URLs	0.9477	0.9428	0.9740	0.9898	0.9887	0.9990
− encabezado y créditos	0.9469	0.9418	0.9733	0.9895	0.9883	0.9990

Tabla 24: Desempeño en Dev al eliminar progresivamente las marcas de la fuente.

Interpretación. Al eliminar todas las marcas, el F_1 de la regresión logística baja de 0.9935 a 0.9883 y el AUC de 0.9996 a 0.9990; el de Bayes ingenuo baja de 0.9491 a 0.9418. La caída es pequeña en términos de F_1, pero el número de errores de la regresión logística casi se duplica (de ≈ 34 a ≈ 61 de 5,797 noticias de Dev). Concluimos que:

- las marcas explícitas contribuyen al desempeño, pero **el modelo no depende solo de ellas**;
- persisten otras señales redundantes de fuente, estilo y tema (por ejemplo, *said* y los nombres propios de eventos, o nombres de sitios como *21wire*), que esta limpieza no elimina;
- por lo tanto, las métricas altas **no deben interpretarse como evidencia de que los modelos detectan la veracidad** de una noticia, sino la fuente y el tema con los que está asociada en este corpus.

8. Debate: modelo generativo frente a modelo discriminativo

8.1. Argumento a favor del modelo generativo (Bayes ingenuo)

- **Simplicidad y costo.** Lo estimamos contando, con una fórmula cerrada y sin optimización iterativa. Su único hiperparámetro, α, es poco sensible: el F_1 en Dev solo varía entre 0.9492 y 0.9510 para $\alpha \in [0.01, 2]$.
- **Interpretabilidad directa.** Sus pesos son log-razones calculadas de los conteos. Gracias a ello, en el Control 1 pudimos exponer de inmediato que las señales dominantes eran artefactos de fuente (*https*, *21wire*) y no de veracidad.
- **Es un excelente *baseline*.** Sin ajustes complejos alcanza un F_1 de 0.9467 en Test, muy por encima del piso de exactitud de 0.5484.
- **Régimen de pocos datos.** La literatura [4] señala que el generativo converge más rápido con pocos datos. *No lo verificamos en este proyecto*: con 27,052 documentos de entrenamiento no estamos en ese régimen.

8.2. Argumento a favor del modelo discriminativo (regresión logística)

- **Mejor desempeño en todas las métricas.** En Test, el F_1 sube de 0.9467 a 0.9931 y el AUC de 0.9760 a 0.9995; los errores se reducen de 280 a 36 (≈ 87 %).
- **La ventaja es del modelo, no de la representación.** En la ablación (Tabla 14), con idénticas representaciones, la regresión logística supera a Bayes ingenuo por 4.3 puntos de F_1 con BoW y por 5.2 con TF-IDF.
- **No supone independencia.** Reparte el peso entre palabras correlacionadas en lugar de contar varias veces la misma evidencia.
- **Probabilidades más útiles.** Solo el 1.14 % de las noticias de Dev tiene probabilidad intermedia en Bayes ingenuo, contra 12.01 % en la regresión logística. El umbral de decisión solo es una palanca real en el modelo discriminativo (Sección 5.5).
- **Regularización explícita.** El parámetro C controla el sobreajuste de forma interpretable (Figura 8).

8.3. Conclusión del debate

Concluimos que los resultados favorecen claramente al modelo discriminativo como *clasificador final*: es superior en exactitud, F_1 y AUC, y sus probabilidades nos permiten estudiar los costos de los errores, algo que Bayes ingenuo no permite por su saturación. Su ventaja no se debe a una representación distinta sino al método de estimación. Pero el modelo generativo no pierde su valor: es el *baseline* natural, es barato y, sobre todo, es una excelente herramienta

de diagnóstico, porque sus pesos son directamente legibles y revelaron que el corpus contiene marcas de fuente que cualquier modelo puede explotar.

Una salvedad importante es que evaluamos la comparación en un corpus donde ambos modelos explotan señales de fuente y tema (Sección 7). La diferencia entre ellos es real, pero las cifras absolutas no deben leerse como la capacidad de detectar noticias falsas en general.

9. Conclusiones

1. **Baseline.** Bayes ingenuo multinomial con BoW alcanza en Test un F_1 de 0.9467 y un AUC-ROC de 0.9760.
2. **Modelo discriminativo.** La regresión logística con TF-IDF y regularización L_2 ($C = 100$) alcanza un F_1 de 0.9931 y un AUC-ROC de 0.9995, con 36 errores contra 280 del *baseline*.
3. **Causa de la mejora.** La ablación muestra que proviene del modelo y no de la representación: con los mismos parámetros, la regresión logística supera a Bayes ingenuo por más de 4 puntos de F_1 con ambas representaciones, mientras que cambiar de BoW a TF-IDF modifica el F_1 en menos de un punto.
4. **Umbral.** En Bayes ingenuo el umbral casi no tiene efecto (98.86 % de las probabilidades saturadas). En la regresión logística controla el intercambio entre falsos positivos y falsos negativos, pero con costos comparables el valor 0.5 ya es casi óptimo; ajustarlo con el F_1 de Dev no mejora los resultados en Test.
5. **Atributos.** Longitud, dígitos, URLs y proporción de mayúsculas no mejoran el desempeño (diferencia de F_1 de 0.0000 en Test), porque su información ya está contenida en las palabras o es débil por sí misma.
6. **Controles.** La permutación de etiquetas lleva el AUC a 0.504 y descarta errores del procedimiento. El análisis de log-razones y la eliminación progresiva de marcas muestran que los modelos aprenden *fuente, estilo y tema* y que el desempeño no depende solo de marcas explícitas.

9.1. Limitaciones

- **Validez del objetivo.** En este corpus las clases difieren en fuente y tema: 99.83 % de las noticias verdaderas contiene "Reuters" y la columna `subject` separa las clases sin ningún traslape (no la usamos como atributo). Nuestros resultados no demuestran capacidad de detectar falsedad en general.
- **Duplicados.** Solo eliminamos textos idénticos; no detectamos los casi duplicados.
- **Textos vacíos.** Conservamos 363 noticias de menos de 20 palabras, todas falsas, que podrían enseñarle al modelo que "texto vacío" implica *Fake*.
- **Malla de C.** El mejor valor ($C = 100$) está en el extremo de la malla que exploramos.
- **Costos.** No definimos una razón de costos propia para noticias; en el ajuste del umbral usamos el criterio de F_1.

- **Alcance temporal.** Las fechas visibles en el corpus corresponden a unos pocos años y los términos más asociados a la clase verdadera son temas de la época, por lo que el modelo podría no generalizar a otros periodos.

Referencias

[1] A. L. Berger, S. A. Della Pietra y V. J. Della Pietra. *A maximum entropy approach to natural language processing*. Computational Linguistics, 22(1):39–71, 1996.

[2] D. Jurafsky y J. H. Martin. *Speech and Language Processing*. Capítulos sobre Bayes ingenuo y regresión logística (3.ª ed., borrador).

[3] C. D. Manning y H. Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

[4] A. Y. Ng y M. I. Jordan. *On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes*. Advances in Neural Information Processing Systems 14, 2002.

[5] C. M. Ocampo Ramírez, R. S. Castillo Ortega y K. Vargas Flores. *Proyecto Análisis de Noticias Falsas* (`Project_FakeNews.ipynb`). Cuaderno de Colab, 2026. https://github.com/kao05/SpamAnalysis-TextMining/blob/main/Project_FakeNews.ipynb

[6] F. Pedregosa et al. *Scikit-learn: machine learning in Python*. Journal of Machine Learning Research, 12:2825–2830, 2011.

[7] G. K. Zipf. *Human Behavior and the Principle of Least Effort*. Addison-Wesley, 1949.

A. Bitácora de contribuciones

Etapa	Integrante	Rol / tarea	Revisó
1	César	Formalización en papel	Rebecca
1	Rebecca	Implementación	Kevin
1	Kevin	Evaluación (métricas y control)	César
2	César	TF-IDF + regresión logística con regularización L_2	Rebecca
2	Kevin	Comparar ambos modelos	César
2	Rebecca	Estudiar el efecto del umbral de decisión	Kevin
3	Rebecca	Añadir atributos (longitud, dígitos, URLs, proporción de mayúsculas)	César
3	Kevin	Ajustar el umbral	Rebecca
3	César	Evaluación (experimento control y métricas)	Kevin

Tabla 25: Bitácora de contribuciones por etapa. Quien implementa un componente no es quien lo revisa.